

Task 07: Containerise and Deploy Automotive AI Agent

AWS ECS Fargate Deployment Report

Rohit

AI Agents and Automation Internship Programme

July 2026

Technologies Used

Docker, FastAPI, Amazon ECR, Amazon ECS Fargate,
Application Load Balancer, AWS Certificate Manager, AWS VPC,
GitHub Actions, AWS CloudFormation, AWS Secrets Manager, CloudWatch

Contents

1 Task Overview	4
2 Project Architecture	4
3 FastAPI Application	5
4 Automotive AI Agent	6
4.1 Structured Database Queries	6
4.2 Graph-Ranked Search	6
5 Docker Containerization	7
5.1 Docker Image Validation	8
6 Docker Dependencies	8
7 Amazon Elastic Container Registry	8
8 ECR Lifecycle Policy	9
9 GitHub Actions CI Pipeline	9
10 AWS Secrets Manager	10
11 CloudFormation Infrastructure	11
12 Private Subnet Deployment	12
13 Application Load Balancer	12
13.1 HTTPS Configuration	13
14 CloudWatch Logging	13
15 Deployment Issues and Resolutions	14
15.1 Anthropic Authentication Error	14

15.2 Target Group Replacement Error	14
15.3 Missing SciPy Dependency	15
15.4 Local Environment API Key Conflict	15
16 Live Endpoint Testing	15
17 Task 06 and ECS Query Comparison	16
18 Final Deployment Details	18
19 Screenshot Evidence	18
20 Security Considerations	19
21 Cost Considerations	20
22 Lessons Learned	20
23 Conclusion	21
A Repository Structure	21
B Useful Commands	22
B.1 Run Locally	22
B.2 Build Docker Image	22
B.3 Validate CloudFormation	22
B.4 Deploy CloudFormation	22
B.5 View ECS Logs	23
B.6 Delete Stack After Review	23

1. Task Overview

The objective of Task 07 was to transform the Automotive AI Agent developed in Task 06 into a production-style cloud application. In Task 06, the agent was executed locally and interacted with automotive data through structured database queries and graph-based ranking methods. In Task 07, the same agent was containerized using Docker and deployed as a live web service on Amazon Web Services.

The deployment was designed around a FastAPI application that exposes the agent through REST API endpoints. The application was packaged as a Docker image, stored in Amazon Elastic Container Registry, and deployed using Amazon Elastic Container Service with the Fargate launch type.

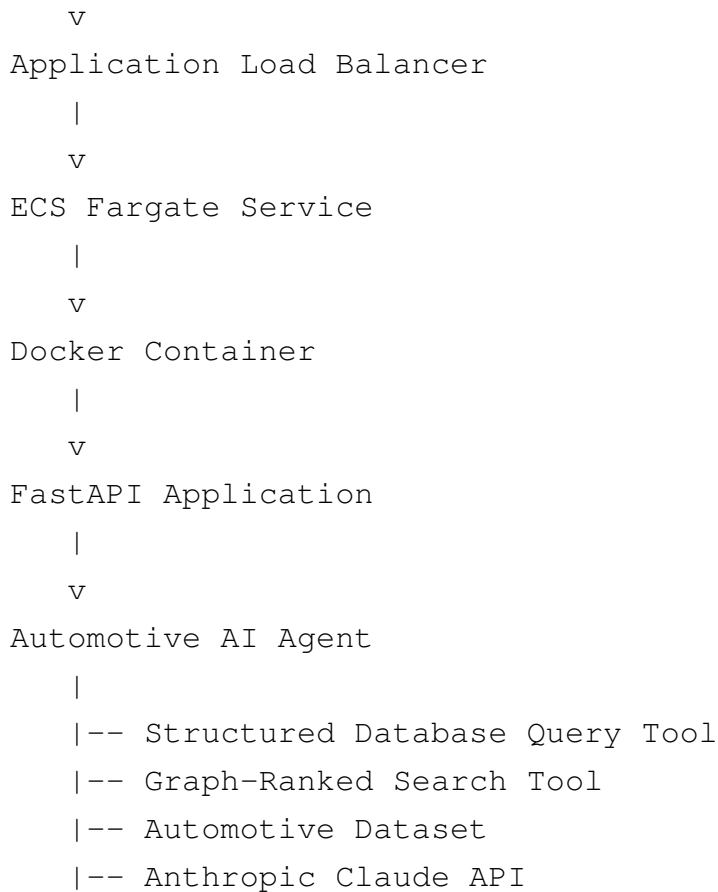
The main objectives of this task were:

- Create a production-grade Docker image.
- Automate Docker image building and pushing using GitHub Actions.
- Store versioned images in Amazon ECR.
- Deploy the container using Amazon ECS Fargate.
- Place ECS tasks in private subnets behind an Application Load Balancer.
- Store API keys securely in AWS Secrets Manager.
- Provision infrastructure using AWS CloudFormation.
- Test the deployed endpoint using the same ten queries from Task 06.
- Compare local and deployed responses.
- Configure semantic-version and Git SHA image tags.
- Configure an ECR lifecycle policy.

2. Project Architecture

The system follows a production-style cloud architecture.

```
User
  |
  | HTTP/HTTPS Request
```



The Application Load Balancer accepts incoming requests and forwards them to the ECS Fargate task. The ECS task runs in private subnets and does not have a public IP address. A NAT Gateway provides outbound internet access so that the container can communicate with the Anthropic API and required AWS services.

3. FastAPI Application

The local Task 06 agent was wrapped in a FastAPI application. This converted the command-line agent into a web service that can receive requests through HTTP.

The application exposes the following endpoints:

- GET / – Returns basic application information.
- GET /health – Returns the application health status.
- POST /query – Accepts a user question and returns the AI agent response.

An example request body is shown below:

```
{  
  "question": "Show me Ferrari cars above 700 horsepower"  
}
```

The `/query` endpoint calls the Task 06 agent and returns a JSON response containing the original question and the generated answer.

4. Automotive AI Agent

The Automotive AI Agent supports two main retrieval methods.

4.1. Structured Database Queries

The structured database query tool is used for questions involving direct filters such as:

- Brand
- Horsepower
- Fuel type
- Price
- Top speed
- Year

For example:

```
Show me Ferrari cars above 700 horsepower
```

The agent identifies this as a structured filtering request and calls the database query tool.

4.2. Graph-Ranked Search

The graph-ranked search tool is used for questions involving terms such as:

- Important
- Influential

- Central
- Graph-ranked

The automotive similarity graph is analyzed using PageRank and degree centrality. Vehicles with stronger network connections receive higher importance scores.

For example:

```
Show important Porsche cars using graph
```

The agent selects the graph-ranked search tool and returns the most central Porsche vehicles.

5. Docker Containerization

A production-grade Dockerfile was created to package the FastAPI application and all dependencies.

The Docker configuration includes:

- Multi-stage image build.
- Lightweight Python base image.
- Non-root application user.
- Dependency installation through `requirements.txt`.
- Minimal runtime image.
- Correct application working directory.
- FastAPI application startup through Uvicorn.

The container starts the API using:

```
uvicorn app.main:app --host 0.0.0.0 --port 8000
```

A `.dockerignore` file prevents unnecessary and sensitive files from being copied into the image. Excluded files include:

- `.env`
- `.git`

- Python cache directories
- Local virtual environments
- Temporary files

This ensured that the Anthropic API key was not stored inside the Docker image.

5.1. Docker Image Validation

The Docker image was successfully built locally using Docker Desktop.

The build completed without errors using the production multi-stage Dockerfile and generated the runtime image used for deployment to Amazon ECS Fargate.

The container was also validated within the GitHub Actions workflow before publishing to Amazon ECR.

6. Docker Dependencies

The main application dependencies include:

```
fastapi
uvicorn
anthropic
python-dotenv
pandas
networkx
scipy
```

The `scipy` dependency was added because NetworkX PageRank requires SciPy for the deployed graph-ranking implementation.

Without SciPy, graph-based queries produced the following error:

```
ModuleNotFoundError: No module named 'scipy'
```

After adding SciPy, the image was rebuilt and graph queries worked successfully.

7. Amazon Elastic Container Registry

Amazon ECR was used to store the Docker images.

The repository name is:

```
task07-agent
```

The repository URI is:

```
654999855057.dkr.ecr.us-east-1.amazonaws.com/task07-agent
```

The final deployed image was:

```
654999855057.dkr.ecr.us-east-1.amazonaws.com/task07-agent:v1.0.6
```

The same image was also tagged using the Git commit SHA:

```
ceac525243bdbf290b7368f7a85c3418e9106e9f
```

This satisfies the requirement to use both semantic versioning and Git SHA tagging.

8. ECR Lifecycle Policy

An ECR lifecycle policy was configured to remove older container images automatically.

The lifecycle policy prevents unused Docker images from accumulating in the repository. This reduces storage usage and helps maintain a clean image history.

The policy keeps the required recent images while deleting images that exceed the configured retention limit.

9. GitHub Actions CI Pipeline

A GitHub Actions workflow was created in:

```
.github/workflows/deploy.yml
```

The workflow runs whenever code is pushed or merged into the `main` branch.

The workflow performs the following steps:

1. Checks out the repository.
2. Installs Python dependencies.
3. Runs Python syntax checks.
4. Configures AWS credentials.

5. Logs into Amazon ECR.
6. Builds the Docker image.
7. Tests the Docker image.
8. Tags the image using a semantic version.
9. Tags the image using the Git commit SHA.
10. Pushes both tags to Amazon ECR.

The semantic image tag is generated using the GitHub Actions run number.

For example:

```
v1.0.6
```

The same image is also tagged with the full Git SHA.

The workflow builds the Docker image, performs a container validation by compiling the application source code inside the container, tags the image with both a semantic version and Git commit SHA, and pushes both tags to Amazon Elastic Container Registry automatically whenever changes are merged into the main branch.

10. AWS Secrets Manager

The Anthropic API key was stored in AWS Secrets Manager.

The secret name is:

```
task07/anthropic-api-key
```

The secret is stored as JSON:

```
{
  "ANTHROPIC_API_KEY": "hidden-value"
}
```

The ECS task definition injects the secret as an environment variable.

The CloudFormation configuration uses:

```
Secrets:
- Name: ANTHROPIC_API_KEY
  ValueFrom: !Sub
    "${AnthropicSecretArn}:ANTHROPIC_API_KEY::"
```

The API key is not hardcoded in:

- Python source code
- Dockerfile
- Docker image
- CloudFormation template
- GitHub repository

11. CloudFormation Infrastructure

AWS CloudFormation was used to provision the infrastructure.

The stack name is:

```
task07-agent-stack
```

The CloudFormation template creates or configures:

- ECS cluster
- ECS task definition
- ECS Fargate service
- Application Load Balancer
- Target group
- HTTP listener
- ECS execution role
- CloudWatch log group
- Security groups
- Private subnets
- Private route table
- NAT Gateway
- Elastic IP address

The infrastructure definition is stored in:

```
cloudformation/ecs-fargate.yml
```

The CloudFormation template accepts deployment parameters for the Docker image URI, Secrets Manager ARN, networking resources, application domain name, and ACM certificate ARN. This allows the same infrastructure template to be reused across deployments without modifying the template source code.

12. Private Subnet Deployment

The ECS Fargate tasks were deployed in private subnets.

The ECS service network configuration confirms:

```
"assignPublicIp": "DISABLED"
```

The private subnet IDs are:

```
subnet-0df6038a41b13edcd  
subnet-0cb41b5a13b2d7bf2
```

The Application Load Balancer remains in public subnets and forwards traffic to the private ECS tasks.

The private tasks use a NAT Gateway for outbound communication with the Anthropic API.

13. Application Load Balancer

The Application Load Balancer provides the public entry point for the deployed service.

The current ALB DNS name is:

```
task07-agent-alb-1856949895.us-east-1.elb.amazonaws.com
```

The live application endpoint is:

```
https://testagent.cciplatform-ai.com
```

The target group health check uses:

```
GET /health
```

A successful health response is:

```
{  
  "status": "healthy"  
}
```

13.1. HTTPS Configuration

The Application Load Balancer was configured with HTTPS termination using AWS Certificate Manager (ACM).

A company-provided domain:

```
testagent.cciplatform-ai.com
```

was associated with an ACM certificate issued in the `us-east-1` region.

An HTTPS listener was added on port 443 and configured to forward requests to the ECS target group.

The application is therefore accessible using both:

- HTTP (port 80)
- HTTPS (port 443)

The HTTPS endpoint provides encrypted communication between clients and the Application Load Balancer while traffic between the load balancer and ECS service remains within the AWS VPC.

14. CloudWatch Logging

The ECS container sends application logs to Amazon CloudWatch.

The log group is:

```
/ecs/task07-agent
```

CloudWatch logs were used to diagnose deployment issues, including:

- Invalid Anthropic API key usage.
- Missing SciPy dependency.

- Graph query exceptions.
- HTTP request status codes.
- Load balancer health checks.

The logs confirmed successful structured and graph-based requests after the fixes were deployed.

15. Deployment Issues and Resolutions

15.1. Anthropic Authentication Error

Initially, the deployed `/query` endpoint returned:

```
500 Internal Server Error
```

CloudWatch showed:

```
anthropic.AuthenticationError: invalid x-api-key
```

The issue was caused by:

```
load_dotenv(env_path, override=True)
```

This allowed a `dotenv` value to override the environment variable injected by ECS.

The code was changed to:

```
load_dotenv(env_path, override=False)
```

After rebuilding and redeploying the image, the ECS-injected Secrets Manager key was used successfully.

15.2. Target Group Replacement Error

A CloudFormation update initially failed because the target group had a hardcoded name:

```
Name: task07-agent-tg
```

CloudFormation attempted to create a replacement target group before deleting the existing one, producing:

```
target group task07-agent-tg already exists
```

The fixed physical name was removed so CloudFormation could generate a unique replacement name.

15.3. Missing SciPy Dependency

Graph queries initially failed with:

```
ModuleNotFoundError: No module named 'scipy'
```

The dependency was added to `requirements.txt`, and a new image was built and deployed as version `v1.0.6`.

15.4. Local Environment API Key Conflict

The local application initially returned an authentication error because a Windows environment variable contained a different API key from the local `app/.env` file.

The session variable was cleared using:

```
Remove-Item Env:ANTHROPIC_API_KEY
```

The local application then loaded the correct key from `app/.env`.

16. Live Endpoint Testing

The deployed health endpoint was tested using PowerShell:

```
Invoke-RestMethod `
    -Uri "https://testagent.cciplatform-ai.com/health"
```

The response was:

```
status
-----
healthy
```

A query was tested using:

```
$body = @{
    question = "Show me Ferrari cars above 700 horsepower"
} | ConvertTo-Json

Invoke-RestMethod `
```

```
-Uri "https://testagent.cciplatform-ai.com/query" `
-Method Post `
-ContentType "application/json" `
-Body $body
```

The endpoint returned a successful answer with HTTP status code 200.

17. Task 06 and ECS Query Comparison

The same ten queries were executed against:

- The local FastAPI prototype.
- The deployed ECS Fargate endpoint.

The local endpoint was:

```
http://127.0.0.1:8000/query
```

The ECS endpoint was:

```
https://testagent.cciplatform-ai.com/query
```

#	Query	Local	ECS	Match	Notes
1	Show me Ferrari cars above 700 horsepower	Successful	Successful	Yes	Same Ferrari records and horsepower filter.
2	Show Porsche vehicles	Successful	Successful	Yes	Same structured Porsche results.
3	Show hybrid vehicles	Successful	Successful	Yes	Same fuel-type filtering and main records.
4	Show cars between 500 and 700 horsepower	Successful	Successful	Yes	Same horsepower range and major results.
5	Show Rolls-Royce cars above 700 horsepower	No records	No records	Yes	Both returned the same no-result conclusion.

#	Query	Local	ECS	Match	Notes
6	Show important Porsche cars using graph	Successful	Successful	Yes	Same graph tool and similar PageRank ordering.
7	Show important Ferrari cars using graph	Successful	Successful	Yes	Same central Ferrari vehicles and ranking method.
8	Show influential BMW vehicles	Successful	Successful	Yes	Same graph-ranked BMW vehicles.
9	Show central Mercedes-Benz vehicles	Successful	Successful	Yes	Same centrality-based Mercedes-Benz results.
10	Show high-performance Lamborghini vehicles using graph	Successful	Successful	Yes	Same graph-ranked Lamborghini vehicles.

The natural-language wording varied slightly between local and ECS responses because large language model outputs are nondeterministic. However, the selected tools, vehicle records, graph-ranking approach, and overall conclusions were consistent.

18. Final Deployment Details

AWS Region	us-east-1
ECR Repository	task07-agent
CloudFormation Stack	task07-agent-stack
ECS Cluster	task07-agent-cluster
ECS Service	task07-agent-service
Application Load Balancer	task07-agent-alb
Secrets Manager Secret	task07/anthropic-api-key
CloudWatch Log Group	/ecs/task07-agent
Public Domain	testagent.cciplatform-ai.com
Protocols	HTTP + HTTPS
Docker Build	Multi-stage
Application User	Non-root
GitHub Actions	Enabled

Table 2: Final Deployment Configuration

19. Screenshot Evidence

Place the screenshot files inside a folder named `Screenshots` in the Overleaf project.

Uncomment each figure after the corresponding screenshot has been uploaded.

Recommended screenshot names:

1. 01_dockerfile.png
2. 02_dockerignore.png
3. 03_github_actions_success.png
4. 04_ecr_version_tags.png
5. 05_ecr_lifecycle_policy.png
6. 06_secrets_manager.png
7. 07_cloudformation_stack.png
8. 08_ecs_cluster.png
9. 09_ecs_service.png

10. 10_ecs_running_task.png
11. 11_ecs_private_networking.png
12. 12_application_load_balancer.png
13. 13_target_group_healthy.png
14. 14_health_endpoint.png
15. 15_structured_query.png
16. 16_graph_query.png
17. 17_local_query.png
18. 18_mentor_signoff.png
19. 19_https_endpoint.png

20. Security Considerations

The following security measures were implemented:

- The application runs as a non-root Docker user.
- The Anthropic API key is stored in Secrets Manager.
- The API key is injected at runtime.
- The local `.env` file is excluded from Docker.
- The ECS task has no public IP address.
- Only the Application Load Balancer can reach port 8000 on the ECS task.
- The ECS execution role has limited Secrets Manager permissions.
- Docker images are stored in a private ECR repository.
- HTTPS encryption using AWS Certificate Manager.
- TLS termination at the Application Load Balancer.

21. Cost Considerations

The deployment uses AWS resources that may generate ongoing charges, including:

- ECS Fargate task
- Application Load Balancer
- NAT Gateway
- Elastic IP address
- CloudWatch logs
- Secrets Manager
- ECR image storage

The NAT Gateway and Application Load Balancer generate hourly charges while active. After the assignment is reviewed and signed off, the CloudFormation stack should be deleted if the live endpoint is no longer required.

22. Lessons Learned

This task provided practical experience with production deployment and cloud engineering.

The main lessons learned were:

- How to convert a local AI agent into a web API.
- How to create a secure production Docker image.
- How to deploy containers using ECS Fargate.
- How to use private subnets and NAT Gateway routing.
- How to configure an Application Load Balancer.
- How to store secrets securely in AWS Secrets Manager.
- How to automate builds and image publishing using GitHub Actions.
- How to use CloudFormation for infrastructure as code.
- How to debug container errors using CloudWatch.
- How to version Docker images with semantic tags and Git SHA tags.

23. Conclusion

Task 07 successfully transformed the Task 06 Automotive AI Agent from a local prototype into a live cloud-hosted service.

The application was containerized with Docker, published to Amazon ECR, and deployed using ECS Fargate. GitHub Actions automated image building, testing, semantic versioning, Git SHA tagging, and image publishing. AWS Secrets Manager securely provided the Anthropic API key to the container at runtime.

The ECS task was deployed in private subnets with public IP assignment disabled. The public Application Load Balancer routed requests to the private Fargate task, AWS Certificate Manager provided HTTPS encryption, and CloudWatch provided centralized application logging.

All ten Task 06 test queries were executed against both the local prototype and deployed ECS endpoint. The results were consistent, with only minor natural-language differences.

Overall, the project demonstrates a production-ready deployment workflow for an AI-powered FastAPI application using Docker, Amazon ECS Fargate, GitHub Actions CI/CD, AWS CloudFormation, AWS Secrets Manager, Application Load Balancer with HTTPS termination, private networking, and secure container deployment practices.

A. Repository Structure

```
TASK 7/  
|  
|-- app/  
|   |-- agent.py  
|   |-- main.py  
|   |-- db_tools.py  
|   |-- graph_search.py  
|   |-- build_graph.py  
|   |-- evaluate_centrality.py  
|   |-- final_csv.csv  
|   |-- centrality_comparison.csv  
|   |-- .env.example  
|  
|-- cloudformation/  
|   |-- ecs-fargate.yml  
|
```

```
|-- .github/  
|   '-- workflows/  
|       '-- deploy.yml  
|  
|-- Dockerfile  
|-- .dockerignore  
|-- requirements.txt  
|-- ecr-lifecycle-policy.json  
'-- README.md
```

B. Useful Commands

B.1. Run Locally

```
uvicorn app.main:app --reload
```

B.2. Build Docker Image

```
docker build -t task07-agent:latest .
```

B.3. Validate CloudFormation

```
aws cloudformation validate-template `--template-body file://cloudformation/ecs-fargate.yml `--region us-east-1 `--no-cli-pager
```

B.4. Deploy CloudFormation

```
aws cloudformation deploy `--template-file cloudformation/ecs-fargate.yml `--stack-name task07-agent-stack `--capabilities CAPABILITY_NAMED_IAM `--region us-east-1 `--parameter-overrides `ImageUri=654999855057.dkr.ecr.us-east-1.amazonaws.com/task07-agent:v1.0.6 `
```

```
AnthropicSecretArn=arn:aws:secretsmanager:us-east-1:654999855057:
secret:task07/anthropic-api-key-LEECmr
```

B.5. View ECS Logs

```
aws logs tail /ecs/task07-agent `
--since 10m `
--region us-east-1 `
--no-cli-pager
```

B.6. Delete Stack After Review

```
aws cloudformation delete-stack `
--stack-name task07-agent-stack `
--region us-east-1
```